



Pontificia Universidad
JAVERIANA
Colombia

Departamento de Ingeniería de Sistemas

Informe de Laboratorio

Comparación TCP/UDP y análisis de concurrencia en Java

Integrantes:

Guillermo Aponte
Ana Sofía Arboleda
Samuel Pico
Daniel Ramírez

Asignatura: Introducción a los Sistemas Distribuidos

Docente: John Jairo Corredor Franco

Fecha: 8 de mayo de 2026

Índice general

Introducción	2
Objetivos	3
1. Comparación entre UDP y TCP	4
1.1. ¿Qué es un socket?	4
1.2. ¿Qué es TCP?	4
1.3. ¿Qué es UDP?	4
1.4. Archivos revisados	5
1.5. Funcionamiento del ejercicio TCP	5
1.6. Funcionamiento del ejercicio UDP	6
1.7. Comparación TCP y UDP	8
1.8. Pruebas realizadas	8
1.9. Observaciones del capítulo	8
2. Identificación y análisis de funcionamiento de la carpeta de Threads	9
2.1. Descripción general	9
2.2. ¿Qué es un thread?	9
2.3. Archivos revisados	10
2.4. Funcionamiento identificado	11
2.5. Tipos de ejecución identificados	12
2.5.1. Ejecución secuencial con Main.java	12
2.5.2. Ejecución concurrente con MainRunnable.java	13
2.6. Comparativa de las tres formas	14
2.7. Comparativa de las ejecuciones realizadas	14
2.8. Diferencia sencilla entre Runnable y Thread	15
2.9. Pruebas realizadas	15
2.10. Observaciones del capítulo	15
Repositorio	16
Conclusiones	17
Referencias	18

Introducción

En este informe se presenta el desarrollo y análisis de dos ejercicios realizados en Java. El primer capítulo corresponde a una comparación entre los protocolos TCP y UDP usando sockets cliente-servidor. El segundo capítulo corresponde al análisis de un conjunto de clases relacionadas con el procesamiento de compras mediante cajeras, donde se identifica el funcionamiento secuencial y dos formas de diseño concurrente usando hilos.

El código fuente completo no se incluye dentro del documento porque se encuentra disponible en el repositorio del proyecto. En este informe se presentan las explicaciones, diferencias, funcionamiento, pruebas realizadas, capturas de pantalla, observaciones y conclusiones.

En el segundo capítulo es importante aclarar que la comparación entre `Runnable` y `Thread` se presenta principalmente como una diferencia de diseño. Para este caso pequeño, donde solo se simula la atención de dos clientes, no es estrictamente necesario tener ambas implementaciones. Sin embargo, se analizan porque permiten comprender dos formas distintas de organizar una ejecución concurrente en Java.

Objetivos

Objetivo general

Analizar el funcionamiento básico de sockets TCP/UDP y de la ejecución secuencial y concurrente en Java, mediante la revisión, ejecución y comparación de programas implementados.

Objetivos específicos

- Comparar las principales características de TCP y UDP.
- Ejecutar programas cliente-servidor para observar el envío de mensajes.
- Identificar el funcionamiento general de un programa con clases, objetos e hilos.
- Diferenciar una ejecución secuencial de una ejecución concurrente.
- Explicar la diferencia de diseño entre usar `Runnable` y extender `Thread`.
- Registrar evidencias mediante capturas de pantalla.
- Presentar conclusiones a partir de las pruebas realizadas.

Capítulo 1

Comparación entre UDP y TCP

1.1. ¿Qué es un socket?

Un socket es un punto de comunicación que permite que dos programas intercambien información a través de una red. Normalmente, un programa actúa como servidor y queda esperando mensajes, mientras que otro programa actúa como cliente y envía información.

En esta práctica se usaron sockets para comparar dos formas de comunicación: TCP y UDP.

1.2. ¿Qué es TCP?

TCP significa *Transmission Control Protocol*. Es un protocolo orientado a conexión. Esto quiere decir que antes de enviar datos, el cliente y el servidor establecen una conexión. Su principal ventaja es que ofrece mayor confiabilidad, ya que busca garantizar que los mensajes lleguen correctamente y en orden.

En el ejercicio, TCP se trabajó con dos archivos:

- `serTCPsocket.java`: servidor TCP.
- `cliTCPsocket.java`: cliente TCP.

1.3. ¿Qué es UDP?

UDP significa *User Datagram Protocol*. Es un protocolo no orientado a conexión. Esto quiere decir que el cliente puede enviar mensajes al servidor sin establecer previamente una conexión formal. Su ventaja es que suele ser más rápido y sencillo, pero no garantiza que todos los mensajes lleguen ni que lleguen en orden.

En el ejercicio, UDP se trabajó con dos archivos:

- `serUDPsocket.java`: servidor UDP.
- `cliUDPsocket.java`: cliente UDP.

1.4. Archivos revisados

Cuadro 1.1: Archivos del capítulo 1

Archivo	Función identificada
<code>serTCPsocket.java</code>	Crea un servidor TCP que espera la conexión de un cliente en el puerto 6001.
<code>cliTCPsocket.java</code>	Crea un cliente TCP que se conecta al servidor y envía mensajes escritos por teclado.
<code>serUDPsocket.java</code>	Crea un servidor UDP que recibe paquetes en el puerto 6000.
<code>cliUDPsocket.java</code>	Crea un cliente UDP que envía mensajes al servidor mediante datagramas.

Pantallazo 1

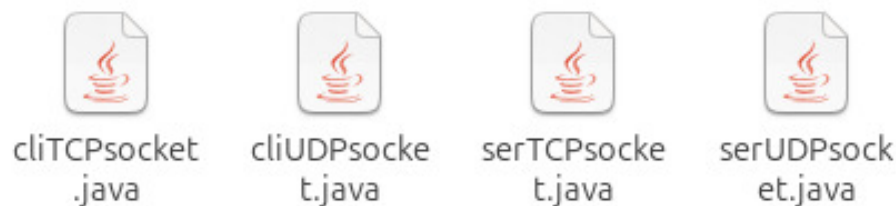


Figura 1.1: Archivos utilizados para la comparación entre TCP y UDP.

1.5. Funcionamiento del ejercicio TCP

El ejercicio TCP funciona de la siguiente manera:

1. Primero se ejecuta el servidor TCP.
2. El servidor queda esperando una conexión en el puerto 6001.
3. Luego se ejecuta el cliente TCP.

4. El cliente se conecta al servidor usando la dirección indicada.
5. El usuario escribe mensajes desde el cliente.
6. El servidor recibe y muestra los mensajes.
7. La comunicación termina cuando se envía la palabra `fin`.

Pantallazo 2

```
sistemas@12C806E14216190:~/Descargas/Taller04/TCP_UDP$ make run-tcp-server
java -cp bin serTCPsocket

==**SOCKETS TCP <<SERVIDOR>>
█
```

Figura 1.2: Servidor TCP esperando y recibiendo mensajes.

Pantallazo 3

```
sistemas@12C806E14216190:~/Descargas/Taller04/TCP_UDP$ make run-tcp-client ARGS=localhost
java -cp bin cliTCPsocket localhost
Prueba de sockets TCP (CLIENTE)
Capturando direccion de host... ok
Creando socket... ok
Introduce mensajes a enviar:
hola
xd
fin
sistemas@12C806E14216190:~/Descargas/Taller04/TCP_UDP$ █
```

Figura 1.3: Cliente TCP enviando mensajes al servidor.

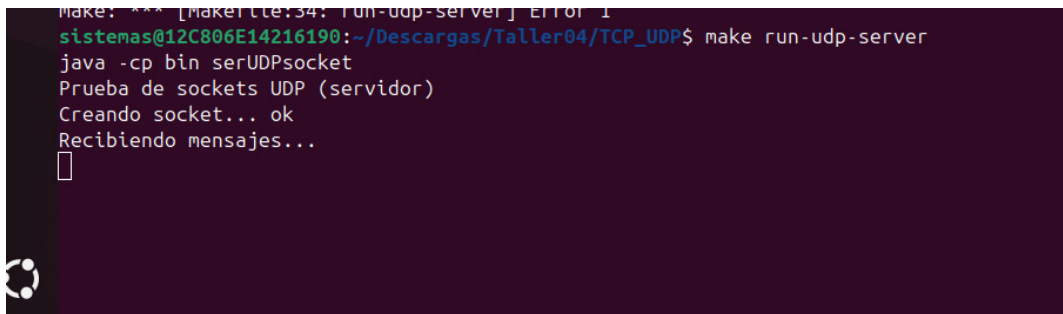
1.6. Funcionamiento del ejercicio UDP

El ejercicio UDP funciona de la siguiente manera:

1. Primero se ejecuta el servidor UDP.
2. El servidor queda esperando datagramas en el puerto 6000.
3. Luego se ejecuta el cliente UDP.

4. El cliente escribe mensajes desde teclado.
5. Cada mensaje se convierte en un paquete o datagrama.
6. El paquete se envía al servidor.
7. El servidor recibe el paquete y muestra el mensaje.
8. La ejecución termina cuando se envía la palabra `fin`.

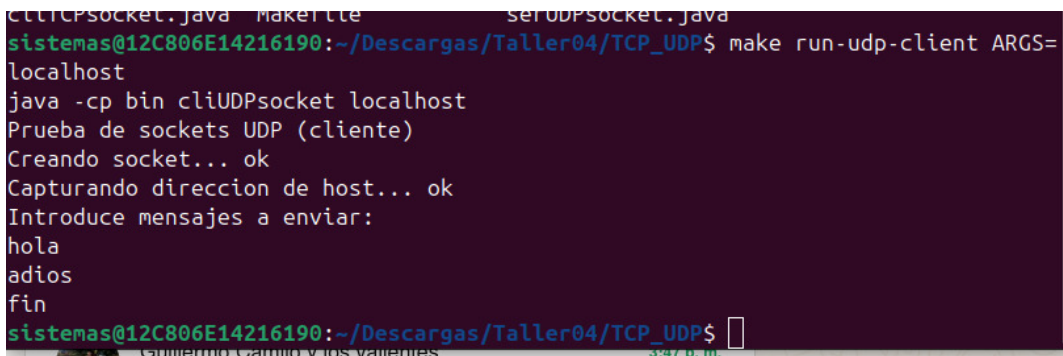
Pantallazo 4



```
make: *** [Makefile:34: run-udp-server] Error 1
sistemas@12C806E14216190:~/Descargas/Taller04/TCP_UDP$ make run-udp-server
java -cp bin serUDPsocket
Prueba de sockets UDP (servidor)
Creando socket... ok
Recibiendo mensajes...
█
```

Figura 1.4: Servidor UDP recibiendo datagramas.

Pantallazo 5



```
cliUDPsocket.java Makefile serUDPsocket.java
sistemas@12C806E14216190:~/Descargas/Taller04/TCP_UDP$ make run-udp-client ARGS=
localhost
java -cp bin cliUDPsocket localhost
Prueba de sockets UDP (cliente)
Creando socket... ok
Capturando direccion de host... ok
Introduce mensajes a enviar:
hola
adios
fin
sistemas@12C806E14216190:~/Descargas/Taller04/TCP_UDP$ █
```

Figura 1.5: Cliente UDP enviando mensajes al servidor.

1.7. Comparación TCP y UDP

Cuadro 1.2: Comparación general entre TCP y UDP

Aspecto	TCP	UDP
Tipo de comunicación	Orientada a conexión	No orientada a conexión
Confiabledad	Mayor	Menor
Velocidad	Menor, porque realiza más control	Mayor, porque es más simple
Orden de mensajes	Mantiene el orden	No garantiza el orden
Forma de envío	Flujo de datos	Datagramas o paquetes
Puerto usado en la práctica	6001	6000
Uso recomendado	Cuando importa que los datos lleguen completos	Cuando importa más la rapidez

1.8. Pruebas realizadas

Cuadro 1.3: Pruebas del capítulo 1

Prueba	Acción realizada	Resultado esperado
TCP	Enviar mensajes desde el cliente TCP	El servidor TCP recibe y muestra los mensajes.
UDP	Enviar mensajes desde el cliente UDP	El servidor UDP recibe y muestra los datagramas.
Finalización	Enviar la palabra fin	El programa termina o deja de recibir mensajes.
Comparación	Observar ambos protocolos	TCP se comporta como conexión establecida y UDP como envío de paquetes.

1.9. Observaciones del capítulo

- TCP requiere una conexión entre cliente y servidor antes de enviar los mensajes.
- UDP permite enviar mensajes sin conexión previa.
- TCP es más confiable, pero puede ser menos rápido.
- UDP es más simple y rápido, pero no asegura completamente la entrega.
- En la práctica, ambos programas permitieron enviar mensajes desde cliente hacia servidor.

Capítulo 2

Identificación y análisis de funcionamiento de la carpeta de Threads

2.1. Descripción general

En la segunda carpeta se identificó un programa en Java relacionado con el procesamiento de compras por parte de cajeras. El programa usa clases como `Cliente`, `Cajera`, `Main`, `MainRunnable` y `CajeraThread`.

A partir de la revisión de los archivos, se interpreta que el objetivo del código es simular cómo una o varias cajeras procesan la compra de varios clientes. Cada cliente tiene un carro de compra representado por números, y esos números indican el tiempo que tarda cada producto en ser procesado.

En este capítulo se diferencian tres formas: ejecución secuencial, ejecución concurrente con `Runnable` y diseño concurrente extendiendo `Thread`. Sin embargo, para este ejercicio concreto, la diferencia entre `Runnable` y `Thread` es principalmente de diseño, no de necesidad práctica. Como solo se atienden dos clientes en una simulación sencilla, con una sola forma concurrente sería suficiente para demostrar la reducción del tiempo. La presencia de ambas formas sirve más para comparar maneras de organizar el código en Java.

2.2. ¿Qué es un thread?

Un *thread* o hilo es una unidad de ejecución dentro de un programa. Permite que varias tareas puedan ejecutarse de manera concurrente. En este caso, los hilos permiten simular que varias cajeras atienden clientes al mismo tiempo.

2.3. Archivos revisados

Cuadro 2.1: Archivos del capítulo 2

Archivo	Función identificada
Cliente.java	Define el cliente y su carro de compra.
Cajera.java	Define la acción de procesar la compra de un cliente.
Main.java	Ejecuta el procesamiento de forma secuencial.
MainRunnable.java	Ejecuta el procesamiento usando Runnable e hilos.
CajeraThread.java	Presenta una alternativa de diseño donde la cajera hereda de Thread.

Pantallazo 6

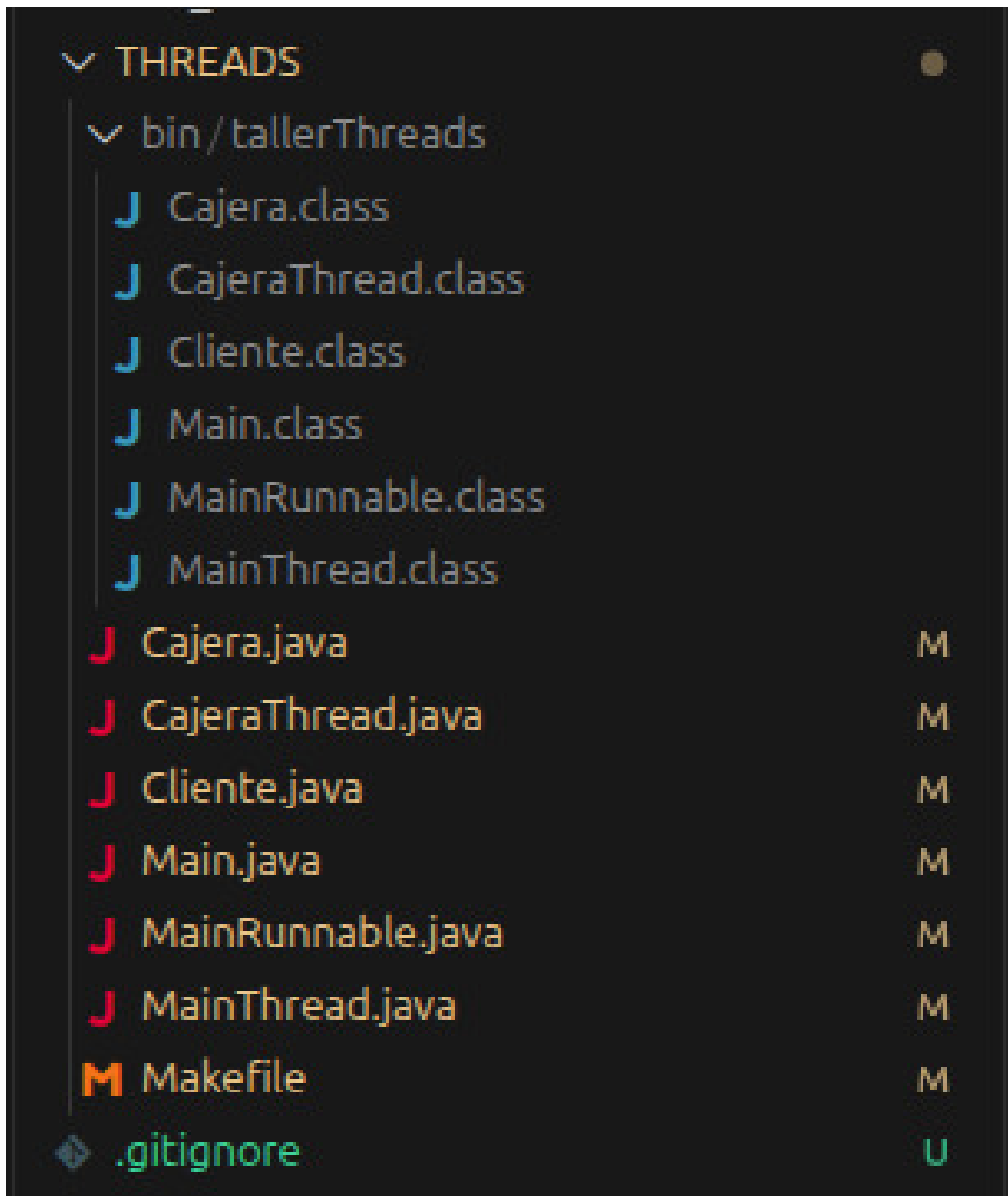


Figura 2.1: Archivos utilizados para el análisis de concurrencia con cajeras.

2.4. Funcionamiento identificado

El programa funciona como una simulación de atención en una caja de supermercado.

1. Se crean dos clientes.
2. Cada cliente tiene un arreglo de números.

3. Cada número representa el tiempo que tarda en procesarse un producto.
4. Se crean una o varias cajeras.
5. Cada cajera procesa los productos de un cliente.
6. El programa muestra en consola el tiempo en el que inicia y termina cada proceso.

Por ejemplo, si un cliente tiene el arreglo:

$$\{2, 2, 1, 5, 2, 3\}$$

significa que tiene seis productos y que cada producto tarda 2, 2, 1, 5, 2 y 3 segundos respectivamente. En total, ese cliente tarda aproximadamente 15 segundos en ser atendido.

2.5. Tipos de ejecución identificados

En la carpeta del capítulo 2 se identifican tres formas relacionadas con la ejecución del programa. La primera corresponde a una ejecución secuencial real. La segunda corresponde a una ejecución concurrente práctica usando `Runnable`. La tercera corresponde a una alternativa de diseño usando `Thread`; en este contexto, esta tercera forma no agrega una mejora práctica adicional frente a `Runnable`, sino que muestra otra manera de estructurar el mismo problema.

2.5.1. Ejecución secuencial con `Main.java`

La ejecución secuencial se encuentra en el archivo `Main.java`. En esta versión, primero se procesa completamente la compra del Cliente 1 y solamente después comienza el procesamiento del Cliente 2.

Esto significa que no hay trabajo simultáneo. El programa debe esperar a que termine una compra para iniciar la siguiente.

- Archivo principal: `Main.java`.
- Tipo de ejecución: secuencial.
- Uso de hilos adicionales: no.
- Comportamiento: primero se atiende un cliente y luego el otro.
- Tiempo esperado: aproximadamente 26 segundos.

El tiempo esperado se calcula sumando los tiempos de los productos de ambos clientes:

Cliente 1: $2 + 2 + 1 + 5 + 2 + 3 = 15$ segundos

Cliente 2: $1 + 3 + 5 + 1 + 1 = 11$ segundos

Total secuencial: $15 + 11 = 26$ segundos

Pantallazo 7

```
● sistemas@12C806E02216192:~/Descargas/Taller04/UNI-26-10-ISD-Taller04-Hilos/THREADS$ make secuencial

>>> EJECUCIÓN versión Secuencial: <<<

java -cp bin tallerThreads.Main
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 15seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 16seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 19seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 24seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 25seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 26seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 26seg

● sistemas@12C806E02216192:~/Descargas/Taller04/UNI-26-10-ISD-Taller04-Hilos/THREADS$
```

Figura 2.2: Ejecución secuencial: primero se procesa un cliente completo y luego el otro.

2.5.2. Ejecución concurrente con MainRunnable.java

La ejecución concurrente con `Runnable` se encuentra en el archivo `MainRunnable.java`. En esta versión, se crean dos procesos o tareas y luego cada una se ejecuta dentro de un hilo diferente usando `new Thread(...).start()`.

En esta ejecución, las dos cajeras pueden trabajar al mismo tiempo. Por eso, el tiempo total ya no corresponde a la suma de ambos clientes, sino aproximadamente al tiempo del cliente que más tarda.

- Archivo principal: `MainRunnable.java`.
- Tipo de ejecución: concurrente.
- Mecanismo usado: implementación de `Runnable`.
- Comportamiento: las dos cajeras atienden clientes al mismo tiempo.
- Tiempo esperado: aproximadamente 15 segundos.

En este caso, como el Cliente 1 tarda 15 segundos y el Cliente 2 tarda 11 segundos, ambos procesos avanzan simultáneamente. Por eso, el tiempo total esperado se acerca a 15 segundos, que corresponde al cliente con mayor tiempo de procesamiento.

Esta implementación sí es práctica para demostrar concurrencia en el ejercicio, porque permite comparar claramente el resultado contra la ejecución secuencial.

Pantallazo 8

```

● sistemas@12C806E02216192:~/Descargas/Taller04/UNI-26-10-ISD-Taller04-Hilos/THREADS$ make runnable

>>> EJECUCIÓN versión Paralela con Runnable: <<<

java -cp bin tallerThreads.MainRunnable
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 11seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 11seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg

● sistemas@12C806E02216192:~/Descargas/Taller04/UNI-26-10-ISD-Taller04-Hilos/THREADS$

```

Figura 2.3: Ejecución concurrente con Runnable: las dos cajeras procesan clientes al mismo tiempo.

2.6. Comparativa de las tres formas

2.7. Comparativa de las ejecuciones realizadas

Cuadro 2.2: Comparación entre la ejecución secuencial y la ejecución con Runnable

Aspecto	Ejecución secuencial	Ejecución con Runnable
Archivo ejecutado	Main.java	MainRunnable.java
Comando usado	make secuencial	make runnable
Tipo de ejecución	Una tarea después de la otra	Dos tareas al mismo tiempo
Uso de hilos	No usa hilos adicionales	Usa Runnable dentro de objetos Thread
Inicio de las cajeras	La Cajera 2 inicia cuando termina la Cajera 1	Las dos cajeras inician aproximadamente al mismo tiempo
Tiempo del Cliente 1	15 segundos	15 segundos
Tiempo del Cliente 2	26 segundos acumulados	11 segundos
Tiempo total observado	26 segundos	15 segundos
Resultado observado	Se procesa primero un cliente y luego el otro	Se procesan los dos clientes de forma simultánea
Conclusión	Es más simple, pero tarda más	Es más eficiente para este caso porque reduce el tiempo total

2.8. Diferencia sencilla entre Runnable y Thread

La diferencia entre `Runnable` y `Thread` puede entenderse de forma simple:

- **Runnable:** la clase es una tarea.
- **Thread:** la clase es el hilo completo.

Con `Runnable`, se define qué se quiere hacer y luego Java crea un hilo para ejecutar esa tarea. Con `Thread`, la clase ya nace siendo un hilo. En este ejercicio, ambas ideas sirven para representar cajeras trabajando al mismo tiempo, pero no era necesario usar las dos para resolver el problema. Se incluyen como comparación para entender las dos posibilidades de diseño.

2.9. Pruebas realizadas

Cuadro 2.3: Pruebas del capítulo 2

Prueba	Acción realizada	Resultado esperado
Revisión de archivos	Identificar clases principales	Se reconocen clientes, cajeras y clases de ejecución.
Ejecución secuencial	Ejecutar <code>Main.java</code>	Un cliente se procesa después del otro.
Ejecución concurrente	Ejecutar <code>MainRunnable.java</code>	Dos clientes se procesan al mismo tiempo.
Revisión de diseño con <code>Thread</code>	Revisar <code>CajeraThread.java</code>	Se identifica una alternativa de diseño para crear hilos.
Comparación de tiempos	Comparar resultados esperados	La versión concurrente tarda menos que la secuencial.

2.10. Observaciones del capítulo

- El código representa una simulación sencilla de atención de clientes por cajeras.
- Los números del carro de compra representan segundos de espera.
- La ejecución secuencial es más simple, pero tarda más.
- La ejecución concurrente con `Runnable` es suficiente para demostrar la mejora de tiempo en esta práctica.
- La clase `CajeraThread` no es estrictamente necesaria para este caso, pero sirve para mostrar otra forma de diseñar hilos en Java.
- En este contexto, la diferencia entre `Runnable` y `Thread` es más de organización del código que de resultado práctico.

Repositorio

El código fuente completo del laboratorio se encuentra en el repositorio personal del grupo. Allí se incluyen los archivos Java correspondientes a la comparación entre TCP y UDP, así como los archivos de la práctica de threads.

Enlaces a los repositorios:

- Guillermo Aponte: https://github.com/guille-011/Sistemas_Distribuidos-Javeriana_2026-10
- Ana Sofía Arboleda: <https://github.com/anarboleda/UNI-26-10-ISD-Taller04-Hilos.git>
- Samuel Pico: <https://github.com/Samu-Kiss/UNI-26-10-ISD-Taller04-Hilos>
- Daniel Ramírez: <https://github.com/CarefreeHarp/Taller04>

Conclusiones

- TCP y UDP permiten comunicar programas mediante sockets, pero funcionan de manera diferente.
- TCP es más confiable porque establece una conexión entre cliente y servidor antes de enviar los datos.
- UDP es más rápido y sencillo porque envía datagramas sin establecer una conexión previa.
- En las pruebas realizadas, ambos protocolos permitieron enviar mensajes desde un cliente hacia un servidor.
- En el segundo ejercicio se identificó una simulación de cajas procesando compras de clientes.
- La ejecución secuencial tarda más porque primero se atiende completamente un cliente y luego el otro.
- La ejecución concurrente con **Runnable** permite que varias tareas avancen al mismo tiempo, reduciendo el tiempo total de procesamiento.
- La implementación con **Thread** se entiende en este contexto como una alternativa de diseño, no como una necesidad práctica, porque el ejercicio ya puede resolverse correctamente usando **Runnable**.
- En esta práctica, la diferencia entre **Runnable** y **Thread** no está tanto en el resultado final, sino en la forma de organizar el código.
- El uso de capturas y repositorio permite evidenciar el desarrollo sin necesidad de pegar todo el código dentro del informe.

Referencias

- IETF. W. Eddy (ed.). *RFC 9293: Transmission Control Protocol (TCP)*. Agosto de 2022. Disponible en: <https://www.rfc-editor.org/rfc/rfc9293.html>
- IETF. J. Postel. *RFC 768: User Datagram Protocol*. 28 de agosto de 1980. Disponible en: <https://www.rfc-editor.org/rfc/rfc768>
- IETF. L. Eggert, G. Fairhurst y G. Shepherd. *RFC 8085: UDP Usage Guidelines*. Marzo de 2017. Disponible en: <https://www.rfc-editor.org/rfc/rfc8085.html>
- Oracle. *Java Documentation*. Disponible en: <https://docs.oracle.com/javase/>
- Oracle. *Class Socket*. Disponible en: <https://docs.oracle.com/javase/8/docs/api/java/net/Socket.html>
- Oracle. *Class ServerSocket*. Disponible en: <https://docs.oracle.com/javase/8/docs/api/java/net/ServerSocket.html>
- Oracle. *Class DatagramSocket*. Disponible en: <https://docs.oracle.com/javase/8/docs/api/java/net/DatagramSocket.html>
- Oracle. *Class DatagramPacket*. Disponible en: <https://docs.oracle.com/javase/8/docs/api/java/net/DatagramPacket.html>
- Oracle. *Class Thread*. Disponible en: <https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html>
- Oracle. *Interface Runnable*. Disponible en: <https://docs.oracle.com/javase/8/docs/api/java/lang/Runnable.html>